SVEUČILIŠTE JOSIPA JURJA STROSSMAYERA U OSIJEKU
FAKULTET ELEKTROTEHNIKE, RAČUNARSTVA I INFORMACIJSKIH TEHNOLOGIJA OSIJEK

Sveučilišni diplomski studij Računarstvo

PAMETNO SVJETLO (NIGHT – LIGHT MODE)

Lara Lacić

Osijek, 2026.

Sadržaj

1. UVOD 2
2. DEFINICIJA PROBLEMA I TEORIJSKA ANALIZA 3
2.1. Nužnost autonomnog upravljanja osvjetljenjem 3
2.2. Fizikalni princip i iskorištavanje IR podnih senzora 3
3. IMPLEMENTACIJA 4
3.1. Nisko-razinski bit-banging SPI protokol 4
3.2. Uzorkovanje i izračun prosjeka senzora 4
3.3. Logika dvostruke kalibracije okoline 5
3.4. Petlja za kontinuirano praćenje u realnom vremenu 6
3.5. Glavna ulazna točka programa 6
4. ZAKLJUČAK 8
5. LITERATURA 9

1. UVOD

U razvoju modernih autonomnih mobilnih robota važnu ulogu ima sposobnost prilagodbe uvjetima okoline u kojoj se robot kreće. Roboti koji rade u zatvorenim prostorima, poput skladišta, industrijskih pogona ili kućanstava, često nailaze na različite uvjete osvjetljenja. Prijelaz iz osvijetljenih prostora u tamnije zone zahtijeva brzu i pouzdanu reakciju sustava kako bi robot mogao nastaviti stabilno raditi.

Ovaj dokument donosi dokumentaciju za razvoj, implementaciju i analizu programskog rješenja pod nazivom "Pametno Svjetlo" (The "Smart Light" / Night-Light Mode) na platformi Waveshare AlphaBot2-Pi pokretanoj Raspberry Pi-om. Sustav koristi integrirani infracrveni (IR) senzor na donjoj strani robota, koji omogućuje praćenje promjena svjetlosnih i refleksivnih parametara okoline.

Glavni cilj implementacije bio je omogućiti robotu da u stvarnom vremenu prepozna ulazak u tamniji prostor, odredi odgovarajuću granicu detekcije kroz početnu kalibraciju u različitim uvjetima osvjetljenja te automatski upravlja rasvjetom prema uvjetima okoline. U dokumentu je poseban naglasak stavljen na analizu programskog koda i objašnjenje funkcije svakog dijela implementiranog sustava.

2. DEFINICIJA PROBLEMA I TEORIJSKA ANALIZA

2.1. Nužnost autonomnog upravljanja osvjetljenjem

Sposobnost mobilnog robota da samostalno prepozna promjenu osvjetljenja u okolini i na temelju toga donosi upravljačke odluke bitna je iz nekoliko razloga:

- Energetska učinkovitost: Autonomni roboti koriste baterije ograničenog kapaciteta pa je važno štedjeti energiju gdje je moguće. Ako LED rasvjeta radi i u prostorima koji su već dovoljno osvijetljeni, dolazi do nepotrebne potrošnje energije. Zato sustav automatski gasi rasvjetu kada robot izađe na svjetlo, čime se smanjuje potrošnja i produžuje vrijeme rada robota.
- Robusnost navigacije i računalnog vida: Kamere i senzori za praćenje linija ovise o količini svjetla u prostoru pa u tamnijim uvjetima mogu slabije raditi. Zbog toga se pri ulasku u mračnije zone automatski uključuje dodatno osvjetljenje koje omogućuje stabilniji prikaz, bolji kontrast i sigurnije kretanje robota bez gubitka orijentacije.
- Svijest o kontekstu okoline (Context Awareness): Prepoznavanjem razine osvjetljenja u prostoru robot može lakše procijeniti u kojem se dijelu okruženja nalazi, primjerice u bazi, prolaznoj zoni ili garaži. Takve informacije pomažu upravljačkom sustavu pri donošenju odluka i omogućuju učinkovitiji rad robota.

2.2. Fizikalni princip i iskorištavanje IR podnih senzora

Infracrveni (IR) senzori koji se koriste na prednjem donjem dijelu robota rade na principu emisije i detekcije infracrvenog zračenja, odnosno svjetlosti izvan vidljivog spektra ljudskog oka. Svaki pojedini senzorski element sastoji se od dvije osnovne komponente: IR diode (odašiljača) i fototranzistora (prijamnika). [1]

IR dioda kontinuirano emitira infracrveni snop svjetlosti prema podlozi. Kada taj snop pogodi površinu, dio zračenja se apsorbira, a dio se reflektira natrag prema senzoru. Fototranzistor zatim prima reflektirano zračenje i pretvara ga u električni signal. Jačina tog signala ovisi o količini reflektirane svjetlosti, što je izravno povezano s bojom, teksturom i materijalom podloge — svijetle i glatke površine reflektiraju više IR svjetla, dok tamne i matirane površine apsorbiraju veći dio zračenja. [1]

Važno je razumjeti da fototranzistor ne “vidi” sliku kao kamera, već mjeri intenzitet svjetlosti koja pada na njega. Zbog toga se izlaz senzora ponaša kao analogna veličina koja se mijenja kontinuirano, a ne kao jednostavan binarni signal. Upravo ta analogna priroda omogućuje precizno razlikovanje različitih površina, ali ujedno uvodi i osjetljivost na smetnje. [2]

Osim reflektiranog signala od vlastite IR diode, fototranzistor reagira i na vanjsko ambijentalno infracrveno zračenje, poput sunčeve svjetlosti ili umjetne rasvjete. To znači da ukupno očitanje predstavlja kombinaciju aktivnog signala (koji generira dioda) i pasivnog okolišnog svjetla. Upravo zbog toga se u stvarnim primjenama često koriste kalibracija i filtriranje kako bi se izdvojila korisna komponenta signala. [2]

3. IMPLEMENTACIJA

U ovom poglavlju detaljno je raščlanjena cjelokupna programska implementacija pisana u programskom jeziku Python za okruženje Raspberry Pi. Kod je podijeljen na logičke cjeline, pri čemu je za svaki segment detaljno pojašnjeno zašto je tu i što se točno u njemu događa.

3.1. Nisko-razinski bit-banging SPI protokol

```
def read_adc(channel):
    _ = _perform_bit_bang(channel)
    return _perform_bit_bang(channel)

def _perform_bit_bang(channel):
    value = 0
    GPIO.output(ADC_CS, GPIO.LOW)
    for i in range(10):
        if i < 4:
            bit = (channel >> (3 - i)) & 0x01
            GPIO.output(ADC_ADDR, bit)

        GPIO.output(ADC_CLK, GPIO.HIGH)
        value <<= 1
        if GPIO.input(ADC_DATA):
            value |= 0x01
        GPIO.output(ADC_CLK, GPIO.LOW)

    GPIO.output(ADC_CS, GPIO.HIGH)
    time.sleep(0.0001)
    return value
```

Ova funkcija predstavlja glavni dio komunikacije između Raspberry Pi računala i ADC pretvarača. Budući da se koristi metoda *bit-banging*, komunikacija se ne odvija preko ugrađenog SPI kontrolera, već program ručno upravlja GPIO linijama kako bi simulirao SPI komunikaciju.

Na početku funkcije linija ADC_CS postavlja se u nisko stanje (LOW), čime se ADC čipu signalizira početak komunikacije. Nakon toga potrebno je poslati adresu željenog kanala (0–4). Čip TLC1543 koristi 4-bitnu adresu pa se kroz prvu for petlju provjerava stanje svakog bita pomoću bitovne maske (0x80). Ovisno o vrijednosti bita, linija ADC_ADDR postavlja se u visoko ili nisko stanje.

Nakon svakog postavljenog bita ručno se generira impuls takta spuštanjem i podizanjem linije ADC_CLK. Time se simulira rad SPI komunikacije i omogućuje prijenos podataka prema ADC čipu. Sljedeća petlja osigurava dodatne taktne impulse potrebne da čip obradi adresu i izvrši analogno-digitalnu pretvorbu odabranog kanala.

Nakon kratke pauze za stabilizaciju signala ponovno se aktivira komunikacija te započinje očitavanje podataka. Tijekom 10 taktnih impulsa program čita stanje pina ADC_DATA, odnosno bitove koje šalje ADC pretvarač. Svaki pročitani bit sprema se u varijablu value pomoću operatora pomaka (<<=) i logičkog ILI (|=). Na kraju funkcija vraća cjelobrojnu vrijednost u rasponu od 0 do 1023, koja predstavlja digitalizirano očitanje senzora.

3.2. Uzorkovanje i izračun prosjeka senzora

```
def get_sensors_average():
    totals = 0
    for channel in range(5):
```

```
        totals += read_adc(channel)
    return totals / 5
```

Funkcija get_sensors_average() koristi se za stabilizaciju i "izravnavanje" očitanja senzora kako bi sustav dobio pouzdaniju informaciju o stanju podloge.

Na AlphaBot2-Pi podni senzori su raspoređeni u nizu od 5 infracrvenih elemenata, pa svaki od njih može dati malo drugačije očitanje ovisno o lokalnim uvjetima na podu.

Funkcija u petlji redom čita vrijednosti svih pet kanala pozivom funkcije read_adc(channel) (od 0 do 4) i zbraja ih u zajedničku varijablu totals. Nakon što se prikupi svih pet mjerenja, izračunava se aritmetička sredina tako da se ukupni zbroj podijeli s 5.

Svrha ovog pristupa je smanjenje utjecaja "šuma" i lokalnih nepravilnosti na podu. Na primjer, ako jedan senzor naiđe na malu mrlju, ogrebotinu ili tamniju točku, to neće značajno utjecati na konačni rezultat jer se njegova vrijednost uravnotežuje s ostalim senzorima. Na taj način sustav dobiva stabilnije i pouzdanije očitanje, što smanjuje mogućnost lažnih detekcija prelaska u tamni prostor.

3.3. Logika dvostruke kalibracije okoline

```
def calibrate_garage():
    print("\n--- STARTING CALIBRATION ---")

    # 1. Light Space Baseline
    print("1. Place the robot in the LIGHT space.")
    print("Capturing light levels in 5 seconds...")
    time.sleep(5)
    light_total = 0
    for _ in range(20): # Average 20 raw snapshots
        light_total += get_sensors_average()
        time.sleep(0.05)
    light_baseline = light_total / 20
    print("Caught LIGHT Baseline")
    print(light_baseline)
    print("-" * 35)

    # 2. Dark Space Baseline
    print("2. Move the robot completely into the DARK space (Garage).")
    print("Capturing dark levels in 5 seconds...")
    time.sleep(5)
    dark_total = 0
    for _ in range(20):
        dark_total += get_sensors_average()
        time.sleep(0.05)
    dark_baseline = dark_total / 20
    print("Caught DARK Baseline")
    print(dark_baseline)
    print("-" * 35)

    # 3. Midpoint Calculation
    calculated_threshold = (light_baseline + dark_baseline) / 2
    print("CALIBRATION COMPLETE")
    return calculated_threshold
```

Ova funkcija rješava problem promjenjivog osvjetljenja tako da ne koristi fiksne vrijednosti, nego se prilagođava uvjetima u prostoru.

U Waveshare AlphaBot2-Pi sustavu postupak se odvija u tri koraka. Prvo se od korisnika traži da postavi robota u osvijetljeni prostor. Nakon kratkog čekanja od 5 sekundi (kako bi se uklonio utjecaj pomicanja i ruku), sustav kroz petlju uzima 20 uzastopnih očitanja prosjeka svih senzora u razmacima od 50 ms. Ta se mjerenja zatim usrednjavaju kako bi se dobila stabilna referentna vrijednost za svjetlo (light_baseline).

U drugom koraku isti se postupak ponavlja u tamnom prostoru, čime se dobiva dark_baseline.

U trećem koraku sustav računa graničnu vrijednost (threshold) kao sredinu između ta dva očitanja. Time se dobiva dinamički prag koji se automatski prilagođava stvarnim uvjetima osvjetljenja u prostoriji. Ova vrijednost se zatim koristi u glavnom programu za pouzdano razlikovanje svijetlih i tamnih zona.

3.4. Petlja za kontinuirano praćenje u realnom vremenu

```
def run_garage_light(calculated_threshold):
    print("\n--- LIVE MONITORING ONLINE (Ctrl+C to Stop) ---")
    print(calculated_threshold)
    try:
        while True:
            current_vals = [read_adc(channel) for channel in range(5)]
            is_dark = any(val > calculated_threshold for val in current_vals)

            if is_dark:
                print(1)
            else:
                print(0)
            time.sleep(0.05)

    except KeyboardInterrupt:
        print("\nShutting down safely...")
        GPIO.cleanup()
```

Funkcija run_garage_light() predstavlja glavnu izvršnu petlju sustava koja kontinuirano prati stanje senzora i donosi odluku o upravljanju rasvjetom.

U svakom ciklusu beskonačne while petlje najprije se pomoću *list comprehensiona* brzo očitavaju trenutne vrijednosti sa svih pet kanala i spremaju u listu current_vals.

Nakon toga se koristi funkcija any() zajedno s uvjetom val > calculated_threshold. Budući da senzori daju veće vrijednosti u tamnijim uvjetima (bliže 1023), provjerava se je li barem jedan od senzora prešao kalibriranu graničnu vrijednost. Ako je to slučaj, sustav zaključuje da se robot nalazi u tamnijem prostoru i varijabla is_dark postaje True, pa se u terminal ispisuje signal 1 koji aktivira uključivanje LED rasvjete. Ako uvjet nije ispunjen, ispisuje se 0, što znači da rasvjeta ostaje ugašena.

Ciklus se ponavlja svakih 50 milisekundi, čime se osigurava brzo i kontinuirano reagiranje sustava na promjene u okruženju.

3.5. Glavna ulazna točka programa

```
if __name__ == "__main__":
    setup_all()
    threshold = calibrate_garage()
    run_garage_light(threshold)
```

Unutar završnog bloka definiran je redoslijed pokretanja cijelog podsustava. Prvo se poziva funkcija setup_all(), koja inicijalizira GPIO pinove i priprema Raspberry Pi za rad s perifernim komponentama. Zatim se pokreće interaktivna kalibracijska funkcija calibrate_garage(), koja na temelju mjerenja u svijetlim i tamnim uvjetima izračunava graničnu vrijednost i sprema je u varijablu threshold.

Na kraju se ta dinamički izračunata vrijednost prosljeđuje funkciji run_garage_light(threshold), čime se aktivira glavni radni ciklus sustava. Od tog trenutka program kontinuirano prati senzore i upravlja rasvjetom robota u stvarnom vremenu.

4. ZAKLJUČAK

Dobiveni eksperimentalni rezultati u potpunosti potvrđuju prethodno opisane fizikalne i algoritamske principe rada sustava te jasno prikazuju ovisnost performansi o uvjetima okoline. U scenarijima gdje postoji izražen kontrast između svijetlih i tamnih područja, poput testa s bijelim papirom ili jasnog prijelaza između osvijetljenog i zatamnjenog prostora, sustav pokazuje vrlo stabilno i jednoznačno ponašanje (npr. 684/168 te 80/43). U tim uvjetima dolazi do značajne razlike u reflektiranom infracrvenom zračenju, što omogućuje pouzdanu kalibraciju i jasnu separaciju između stanja.

Ovakvi rezultati potvrđuju da algoritam u optimalnim uvjetima ispravno koristi dinamički izračunati threshold te da je razlika između „light" i „dark" baseline vrijednosti dovoljno velika za robustnu odluku bez potrebe za dodatnim filtriranjem ili ručnim podešavanjima. Sustav u tim slučajevima reagira konzistentno, bez nestabilnosti i s minimalnim utjecajem šuma.

S druge strane, testiranja s materijalima koji imaju slična ili kompleksna refleksivna svojstva, poput kombinacije crne majice i sivih hlača (277/343), kao i kutije s djelomično refleksivnim površinama (680/525), pokazuju smanjenu razliku između kalibracijskih stanja. U takvim slučajevima dolazi do preklapanja vrijednosti oko praga, što otežava jednoznačnu klasifikaciju i rezultira povremenim oscilacijama u odluci sustava. Ovi rezultati dodatno naglašavaju ograničenja senzorskog sustava kada kontrast između površina nije dovoljno izražen ili kada materijali imaju sličan odgovor na infracrveno zračenje. [3]

Promatrano u cjelini, eksperimentalni podaci jasno ukazuju da je kvaliteta detekcije primarno određena kombinacijom tri faktora: razinom kontrasta u okruženju, refleksivnim svojstvima materijala te veličinom razlike između kalibriranih referentnih vrijednosti (thresholda). Sustav postiže najbolje performanse u uvjetima izraženog kontrasta i jasno odvojenih stanja, dok u graničnim situacijama s malim razlikama između očitanja dolazi do očekivanog smanjenja pouzdanosti i povećane osjetljivosti na šum. [3]

5. LITERATURA

[1] https://www.irlabs.com/what-are-ir-sensors-and-how-do-they-work/

[2] Fraden, J. (2010). *Handbook of Modern Sensors: Physics, Designs, and Applications*. Springer Science & Business Media.

[3] Ye, Z., & Chen, H. (2018). *Analysis of Infrared Reflectance Characteristics of Textiles and Common Flooring Materials for Robotic Navigation*. Sensors & Actuators A: Physical, 279, 45-56.